

Rapport d'Avancement — 22 Avril 2026

Projet : Event Management — Backoffice Angular + Microservices Quarkus

Date : 22 Avril 2026

Auteur : Mariem

1. Intégration SSO Keycloak — Angular 12 Backoffice

1.1 Initialisation Keycloak PKCE

- **Fichier :** `src/app/app.module.ts`
- **Implémentation :** Keycloak initialisé avec `flow: 'standard', pkceMethod: 'S256', onLoad: 'check-sso', silentCheckSsoRedirectUri`
- **Résultat :** Authentification SSO opérationnelle sans rechargement de page complet

1.2 Guard d'authentification par rôle

- **Fichier :** `src/app/core/guards/auth.guard.ts`
- **Implémentation :** `KeycloakAuthGuard` étendu — vérifie l'authentification et les rôles `['admin', 'organizer', 'event-organizer']`
- **Comportement :** Redirige vers la page de login Keycloak si non authentifié

1.3 Service d'authentification

- **Fichier :** `src/app/core/services/auth.service.ts`
- **Implémentation :** `KeycloakService` wrappé avec `BehaviorSubject<currentUser$>` — expose les infos utilisateur (nom, email, rôles, avatar)
- **Méthodes clés :** `login()`, `logout()` → redirige vers `#!/login`, `isAdmin()`, `hasRole()`

1.4 Page Profil utilisateur dynamique

- **Fichiers :** `src/app/pages/user/user.component.ts` + `.html`
- **Avant :** Données statiques hardcodées dans le template
- **Après :** Données réelles lues depuis `authService.currentUser$` (prénom, nom, email, username, rôles)

1.5 Navbar — icône admin conditionnelle

- **Fichier :** `src/app/components/navbar/navbar.component.html`
- **Fix :** Icône admin enveloppée dans `*ngIf="isAdmin()"` — invisible pour les non-admins

1.6 Silent SSO iframe

- **Fichier créé :** `src/assets/silent-check-sso.html`
 - **Rôle :** Page légère chargée dans un iframe invisible pour renouveler le token Keycloak sans redirection
-

2. Démarrage de l'infrastructure microservices

2.1 Services démarrés via Docker Compose

Service	Port	Statut
Zookeeper	2181	✓ Healthy
Kafka	9092	✓ Healthy
MongoDB	27017	✓ Healthy
Eureka Server	8761	✓ Healthy
Config Server	8888	✓ Running
Gateway Service	8080	✓ Running
Keycloak DB	5433	✓ Running
Keycloak	8180	✓ Running

2.2 Problème rencontré — Kafka leader not elected

- **Symptôme** : Kafka ne devenait pas **healthy** lorsque démarré en même temps que Zookeeper
- **Cause** : Race condition entre Zookeeper et Kafka au démarrage
- **Solution** : Démarrage de Kafka séparément après confirmation du statut **healthy** de Zookeeper

3. Débogage — notifications-service crash au démarrage

3.1 Erreur initiale

Au démarrage du conteneur **notifications-service**, crash immédiat avec l'exception :

```
KafkaException: Could not find a public no-argument constructor for
io.quarkus.kafka.client.serialization.ObjectMapperDeserializer
Caused by: java.lang.NoSuchMethodException: ObjectMapperDeserializer.<init>()
```

3.2 Analyse de la cause racine

- **Cause** : **ObjectMapperDeserializer<T>** de **quarkus-kafka-client** est une classe **abstraite** — elle ne peut pas être instanciée directement
- **Configuration incorrecte dans **application.properties**** :

```
# ✗ Avant – classe abstraite référencée directement
mp.messaging.incoming.event-created.value.deserializer=\
  io.quarkus.kafka.client.serialization.ObjectMapperDeserializer
mp.messaging.incoming.event-created.specific.type=\
  com.eventmgmt.notifications.dto.EventMessage
```

- **Problème secondaire** : Utilisation de `JsonbDeserializer` (incompatible avec la version Quarkus 3.8.4)

3.3 Solution — Désérialiseurs concrets

Création de 3 classes concrètes étendant `ObjectMapperDeserializer<T>` :

`EventMessageDeserializer.java`

```
package com.eventmgmt.notifications.dto;
import io.quarkus.kafka.client.serialization.ObjectMapperDeserializer;

public class EventMessageDeserializer extends
ObjectMapperDeserializer<EventMessage> {
    public EventMessageDeserializer() { super(EventMessage.class); }
}
```

`RegistrationMessageDeserializer.java`

```
package com.eventmgmt.notifications.dto;
import io.quarkus.kafka.client.serialization.ObjectMapperDeserializer;

public class RegistrationMessageDeserializer extends
ObjectMapperDeserializer<RegistrationMessage> {
    public RegistrationMessageDeserializer() { super(RegistrationMessage.class); }
}
```

`UserMessageDeserializer.java`

```
package com.eventmgmt.notifications.dto;
import io.quarkus.kafka.client.serialization.ObjectMapperDeserializer;

public class UserMessageDeserializer extends ObjectMapperDeserializer<UserMessage>
{
    public UserMessageDeserializer() { super(UserMessage.class); }
}
```

3.4 Mise à jour `application.properties`

Tous les canaux Kafka entrants mis à jour avec les désérialiseurs concrets :

```
# ✅ Après – désérialiseur concret avec constructeur no-arg
mp.messaging.incoming.event-created.value.deserializer=\
com.eventmgmt.notifications.dto.EventMessageDeserializer

mp.messaging.incoming.registration-created.value.deserializer=\
com.eventmgmt.notifications.dto.RegistrationMessageDeserializer
```

```
mp.messaging.incoming.registration-confirmed.value.deserializer=\
com.eventmgmt.notifications.dto.RegistrationMessageDeserializer

mp.messaging.incoming.user-created.value.deserializer=\
com.eventmgmt.notifications.dto.UserMessageDeserializer
```

4. Rebuild et redémarrage du notifications-service

4.1 Reconstruction de l'image Docker

- Commande : `docker compose build notifications-service`
- Maven télécharge les dépendances Firebase Admin SDK + Google Cloud
- Quarkus augmentation phase complétée
- Seul avertissement non-fatal : `Failed to index MetricRegistry: Class does not exist in ClassLoader`
- Image reconstruite avec succès : `eventmanagement-notifications-service:latest` (16:54:14)

4.2 Démarrage confirmé

```
notifications-service 0.1.0-SNAPSHOT on JVM (powered by Quarkus 3.8.4)
started in 5.686s. Listening on: http://0.0.0.0:8084
```

- Avertissements `LEADER_NOT_AVAILABLE` sur les topics Kafka → normaux (topics créés automatiquement au 1er message)

5. Exploration du notifications-service

5.1 REST API — `http://localhost:8084/api/notifications`

Méthode	Endpoint	Description
GET	/	Lister toutes les notifications
GET	/ {id}	Récupérer une notification par ID
POST	/	Créer et envoyer une notification
PUT	/ {id} /retry	Relancer une notification échouée
DELETE	/ {id}	Supprimer une notification

Swagger UI : `http://localhost:8084/q/swagger-ui`

5.2 Canaux de notification

Canal	Technologie	État
EMAIL	Quarkus Mailer — SMTP (Gmail par défaut, port 587)	✅ Actif
SMS	Twilio SDK 9.14.1	⚙️ Désactivé (<code>twilio.enabled=false</code>)
PUSH	Firebase Admin SDK (FCM)	⚙️ Désactivé (<code>firebase.enabled=false</code>)

5.3 Consommateurs Kafka

Topic	Déclencheur	Traitement actuel
<code>event.created</code>	Nouvel événement créé	Log + TODO envoi email
<code>registration.created</code>	Nouvelle inscription	Log + TODO envoi email
<code>registration.confirmed</code>	Inscription confirmée	Log + TODO envoi email
<code>user.created</code>	Nouvel utilisateur	Log + TODO envoi email

Les handlers Kafka sont fonctionnels (connexion OK) mais la logique d'envoi réel est marquée `// TODO` dans `NotificationConsumer.java`.

5.4 Modèle de données (MongoDB — `notifications_db`)

Champ	Type	Description
<code>recipientId</code>	String	ID du destinataire
<code>type</code>	Enum	EMAIL / SMS / PUSH
<code>message</code>	String	Contenu de la notification
<code>status</code>	Enum	PENDING → SENT / FAILED
<code>sendAt</code>	LocalDateTime	Horodatage d'envoi

6. Débogage — Page Notifications Angular vide

6.1 Problème constaté

La page `localhost:4200/#/notifications` affichait :

- En-tête rouge : *"Failed to load notifications. Please try again."*
- Tableau vide : *"No notifications found"*

6.2 Cause 1 — URL OIDC incorrecte dans Docker

- **Symptôme :** Au démarrage du service, log : `OIDC Server is not available: Connection refused: localhost/127.0.0.1:8180`

- **Cause** : La variable `KEYCLOAK_URL` n'était pas définie dans `docker-compose.yml` pour le `notifications-service` — il utilisait la valeur par défaut `localhost:8180`, inaccessible depuis l'intérieur du conteneur Docker
- **Fix** : Ajout dans `docker-compose.yml` :

```
- KEYCLOAK_URL=http://keycloak:8080/realms/event-mgmt
```

6.3 Cause 2 — Mismatch d'issuer JWT

- **Symptôme** : Le token Angular est émis par `localhost:8180` → `iss=http://localhost:8180/realms/event-mgmt`
- **Cause** : Après correction de l'URL, Quarkus validait le token contre `keycloak:8080` → l'issuer ne correspondait plus → **401 Unauthorized**
- **Fix** : Ajout dans `docker-compose.yml` :

```
- QUARKUS_OIDC_TOKEN_ISSUER=http://localhost:8180/realms/event-mgmt
```

6.4 Cause 3 — Mismatch complet Angular ↔ Backend

- **Symptôme** : Même avec l'API accessible, la table restait vide car les champs du template ne correspondaient pas au modèle backend

Champ Angular (incorrect)	Champ Backend réel
<code>userId</code>	<code>recipientId</code>
<code>read</code> (boolean)	<code>status</code> (SENT/PENDING/FAILED)
<code>createdAt</code>	<code>sendAt</code>
<code>PUT /{id}/read</code>	<code>PUT /{id}/retry</code>

- **Fix** : Mise à jour de 3 fichiers Angular :
 - `notification.service.ts` — interface `Notification` corrigée, `markAsRead()` remplacé par `retryNotification()`
 - `notifications-management.component.ts` — méthode `markAsRead()` remplacée par `retryNotification()`
 - `notifications-management.component.html` — bindings `userId`→`recipientId`, `read`→`status`, `createdAt`→`sendAt`, badges colorés par statut (vert=SENT, orange=PENDING, rouge=FAILED), bouton Retry pour les notifications FAILED

6.5 Données de test insérées

12 notifications insérées directement en MongoDB (`notifications_db`) via `mongosh` :

recipientId	type	message	status
user-001	EMAIL	Registration confirmed for Spring Conference 2026	SENT
user-002	EMAIL	New event: Angular Workshop 2026. Register now!	PENDING
user-001	PUSH	Reminder: Your event starts tomorrow at 10:00 AM.	FAILED

6.6 Résultat final

- API `GET /api/notifications` → `200 OK` avec 12 notifications
- Page Angular rechargée → tableau rempli avec statuts colorés
- Bouton Retry visible uniquement sur les notifications FAILED

7. État des services en fin de session

Service	Port	Statut
Angular Backoffice	4200	✓ Opérationnel (SSO Keycloak actif)
Notifications Service	8084	✓ Opérationnel
Gateway Service	8080	✓ Opérationnel
Eureka Server	8761	✓ Opérationnel
Config Server	8888	✓ Opérationnel
Keycloak	8180	✓ Opérationnel
Kafka	9092	✓ Opérationnel
Zookeeper	2181	✓ Opérationnel
MongoDB	27017	✓ Opérationnel

8. Fichiers modifiés / créés aujourd'hui

Fichier	Action	Description
<code>src/app/app.module.ts</code>	Modifié	Keycloak init PKCE S256 + check-sso
<code>src/app/core/guards/auth.guard.ts</code>	Modifié	KeycloakAuthGuard + vérification rôles
<code>src/app/core/services/auth.service.ts</code>	Modifié	currentUser\$ BehaviorSubject
<code>src/app/pages/user/user.component.ts</code>	Modifié	Données profil depuis Keycloak
<code>src/app/pages/user/user.component.html</code>	Modifié	Bindings dynamiques currentUser\$

Fichier	Action	Description
<code>src/app/components/navbar/navbar.component.html</code>	Modifié	*ngIf isAdmin() sur icône admin
<code>src/assets/silent-check-sso.html</code>	Créé	Iframe silent SSO
<code>notifications-service/.../EventMessageDeserializer.java</code>	Créé	Désérialiseur concret EventMessage
<code>notifications-service/.../RegistrationMessageDeserializer.java</code>	Créé	Désérialiseur concret RegistrationMessage
<code>notifications-service/.../UserMessageDeserializer.java</code>	Créé	Désérialiseur concret UserMessage
<code>notifications-service/src/main/resources/application.properties</code>	Modifié	4 canaux Kafka corrigés
<code>docker-compose.yml</code>	Modifié	KEYCLOAK_URL + QUARKUS_OIDC_TOKEN_ISSUER pour notifications-service
<code>src/app/core/services/notification.service.ts</code>	Modifié	Interface alignée sur modèle backend, retryNotification()
<code>src/app/pages/notifications-management/notifications-management.component.ts</code>	Modifié	retryNotification() remplace markAsRead()
<code>src/app/pages/notifications-management/notifications-management.component.html</code>	Modifié	Bindings corrects + badges statut colorés

9. Points restants / Prochaines étapes

- ☐ Compléter les handlers Kafka dans `NotificationConsumer.java` (remplacer les `// TODO` par l'envoi réel d'emails)
- ☐ Configurer les variables SMTP (`EMAIL_HOST`, `EMAIL_USERNAME`, `EMAIL_PASSWORD`) pour activer l'envoi d'emails
- ☐ Tester le flux complet : création d'un événement → événement publié → email envoyé automatiquement
- ☐ Activer Twilio (SMS) ou Firebase (Push) si besoin
- ☐ Tester les endpoints REST via Swagger UI (<http://localhost:8084/q/swagger-ui>)
- ☐ Démarrer les autres microservices métier (events, registrations, users, charges, dashboard)
- ☒ Fixer l'affichage de la page Notifications (OIDC URL + issuer mismatch + champs Angular) ☒ Résolu

Rapport généré le 22 Avril 2026 — fin de session